

Repaso bucles, for , while y do while

Ejercicios con for

1. Contar hasta 10: Imprime los números del 1 al 10.
2. Números Pares: Imprime todos los números pares del 0 al 20.
3. Suma de Números: Calcula la suma de los números del 1 al 100 e imprímela.

Ejercicios con while

4. Cuenta Regresiva: Imprime una cuenta regresiva desde 5 hasta 1, y luego "¡Despegue!".
5. Adivina el Número (Simple): Simula un juego donde tienes que adivinar un número fijo. El bucle continúa hasta que el número sea adivinado.
6. Validación de Entrada (Básica): Pide al usuario que ingrese una contraseña (simulada) hasta que sea correcta.

Ejercicios con do-while

7. Menú Simple (Una Vez Garantizada): Imprime un menú de opciones al menos una vez, y luego pregunta si el usuario quiere verlo de nuevo. El bucle continúa si el usuario lo desea.
8. Solicitar Número Positivo: Pide al usuario que ingrese un número, y sigue pidiéndolo hasta que ingrese un número positivo.
9. Contador Mínimo: Imprime números del 1 al 3. Asegura que al menos el 1 se imprima, incluso si la condición del while se cumple desde el principio.

Ejercicio Mixto

10. Tablas de Multiplicar (del 1 al 5): Usa un bucle externo (for) para iterar por las tablas del 1 al 5, y un bucle interno (for, while o do-while) para calcular e imprimir cada multiplicación (del 1 al 10).

Empezamos ejercitación de clases

Ejercicio 1: Gestión de Pedidos Online

Descripción: Desarrolla un sistema básico para gestionar pedidos de una tienda online. Cada pedido puede contener múltiples productos.

Requisitos de POO:

- Crea una clase Producto con atributos como nombre (String), precio (double) y cantidadDisponible (int).
- Crea una clase ItemPedido con atributos como producto (objeto Producto) y cantidadSolicitada (int).
- Crea una clase Pedido con un ArrayList de ItemPedido, un idPedido (String) y un estado (String, e.g., "Pendiente", "Completado").
- Diseña el diagrama de clases con los sig. métodos: agregar items, calcular el total del pedido y cambiar el estado.

Aplicación de Bucles:

- **for:** Utiliza un bucle for en la clase Pedido para iterar sobre la lista de ItemPedido y calcular el precio total del pedido.
- **while:** En un método de la clase Pedido, simula la confirmación de disponibilidad de productos. Usa un bucle while para verificar si la

cantidadSolicitada de cada ItemPedido es menor o igual a la cantidadDisponible del Producto asociado. Si no lo es, "solicita" al usuario que ajuste la cantidad hasta que sea válida (simulación de entrada de usuario).

• **do-while:** Implementa un método en la clase Pedido para "procesar" el pedido. Dentro de este método, utiliza un do-while para simular un proceso iterativo de empaquetado de items, decrementando la cantidadDisponible de cada producto hasta que sea cero o se haya procesado la cantidadSolicitada.

Ejercicio 2: Sistema de Votación Electoral

Descripción: Crea un sistema que simule una elección, permitiendo a los votantes elegir entre diferentes candidatos y mostrando los resultados.

Requisitos de POO:

- Define una clase Candidato con atributos como nombre (String), partido (String) y votos (int, inicializado en 0).
- Crea una clase Votante con un idVotante (String) y un booleano yaVoto (inicializado en false).
- Crea una clase Eleccion que contenga un ArrayList de Candidato y un ArrayList de Votante.
- Diseña el diagrama de clases con los sig. métodos: registrar candidatos, registrar votantes, emitir voto y mostrar resultados.

Aplicación de Bucles:

- **do-while:** En la clase Eleccion, implementa un método emitirVoto(Votante votante) que simule el proceso de votación. Usa un do-while para presentar las opciones de candidatos al votante y solicitarle su elección, repitiendo la pregunta si la entrada es inválida o si el votante ya ha votado.
- **for:** Utiliza un bucle for en la clase Eleccion para recorrer la lista de Candidato y mostrar sus nombres y partidos antes de la votación.
- **while:** Después de la votación, usa un bucle while para iterar sobre la lista de Candidato y encontrar al ganador (el que tenga más votos).

Ejercicio 3: Biblioteca Digital

Descripción: Desarrolla un sistema para gestionar libros en una biblioteca digital, permitiendo agregar, buscar y "prestar" libros.

Requisitos de POO:

- Crea una clase Libro con atributos como titulo (String), autor (String), isbn (String), anioPublicacion (int) y disponible (boolean, inicializado en true).
- Crea una clase Usuario con nombre (String), idUsuario (String) y una lista de Libro prestados.
- Crea una clase Biblioteca que contenga un ArrayList de Libro.
- Diseña el diagrama de clases con los sig. métodos: agregar libros, buscar libros por título/autor, prestar libro a un usuario y devolver libro.

Aplicación de Bucles:

- **while:** En la clase Biblioteca, implementa un método buscarLibro(String criterio, String valor) que permita buscar libros. Usa un bucle while para recorrer la lista de libros y agregar aquellos que coincidan con el criterio (título o autor) a una nueva lista que será devuelta.
- **do-while:** En un método prestarLibro(String isbn, Usuario usuario), usa un do-while para simular el intento de préstamo. El bucle podría repetirse si el libro no está disponible o si el ISBN no existe, permitiendo al usuario intentar con otro ISBN (simulación de entrada de usuario).
- **for:** En la clase Usuario, utiliza un bucle for para mostrar los libros que el usuario tiene actualmente prestados.

Ejercicio 4: Sistema de Gestión de Proyectos

Descripción: Desarrolla un sistema simple para gestionar proyectos, permitiendo asignar tareas a diferentes miembros del equipo y seguir su progreso.

Requisitos de POO:

- Crea una clase Tarea con atributos como nombre (String), descripcion (String), estado (String, e.g., "Pendiente", "En Progreso", "Completada") y fechaLimite (String).
- Crea una clase MiembroEquipo con atributos como nombre (String), rol (String) y una lista de Tareas asignadas.
- Crea una clase Proyecto con nombre (String), fechaInicio (String), fechaFinEstimada (String), un ArrayList de Tareas y un ArrayList de MiembroEquipo.
- Diseña el diagrama de clases con los sig. métodos:
agregar tareas, asignar tareas a miembros, actualizar el estado de una tarea y mostrar el progreso del proyecto.

Aplicación de Bucles:

- **for:** En la clase Proyecto, utiliza un bucle for para iterar sobre todas las Tarea y mostrar su estado actual, permitiendo ver el progreso general del proyecto.
- **while:** En un método de la clase Proyecto para asignarTarea(Tarea tarea, MiembroEquipo miembro), usa un bucle while para simular la búsqueda de un miembro del equipo disponible o adecuado para la tarea (podría ser una simulación de criterio o simplemente encontrar el primer miembro disponible).
- **do-while:** En la clase Tarea, crea un método actualizarEstado() que use un do-while para solicitar al usuario el nuevo estado de la tarea (Pendiente, En Progreso, Completada). El bucle debería repetirse si la entrada del usuario no es uno de los estados válidos.